

TALLER PRÁCTICO

Navegación y Hooks en React Native

Aplicación: GameVault – Tienda de Videojuegos

Asignatura: Desarrollo Móvil

Tema: Stack Navigation, Tab Navigation, paso de parámetros, useState, useEffect

Herramientas: React Native CLI, React Navigation, StyleSheet

Entrega: Repositorio Git – rama main

Importante

Todo el código fuente (nombres de variables, funciones, componentes, comentarios en el código) debe estar escrito en inglés. La interfaz de usuario (textos visibles para el usuario) puede estar en español.

1. Objetivo del Taller

Desarrollar una aplicación móvil con React Native CLI llamada GameVault, una tienda de videojuegos que permita al estudiante aplicar de forma integrada los conceptos de navegación (Stack y Tabs), paso de parámetros entre pantallas, y los hooks useState y useEffect.

2. Contexto de la Aplicación

GameVault es una aplicación de catálogo para una tienda de videojuegos. El usuario puede explorar el catálogo de juegos disponibles, consultar el detalle completo de cada título (plataforma, género, precio, descripción), agregar un nuevo videojuego al catálogo mediante un formulario, y acceder a una sección de noticias gaming que se actualiza automáticamente.

3. Arquitectura y Navegación

3.1 Estructura de Navegación

La app debe usar una navegación combinada de Tab Navigation (nivel principal) con Stack Navigation (flujos internos):

Tab	Pantalla(s)	Descripción
Catálogo	GameListScreen	Catálogo de videojuegos disponibles. Cada tarjeta permite navegar al detalle (Stack).
	GameDetailScreen	Recibe parámetros del juego seleccionado vía route.params y muestra plataforma, género, precio, clasificación y descripción.
Agregar	AddGameScreen	Formulario controlado con useState para registrar título, plataforma, género, precio y clasificación de edad del videojuego.
Noticias	GamingNewsScreen	Muestra noticias del mundo gaming. Usa useEffect para simular carga de datos desde una API y un temporizador que rota las noticias automáticamente.

4. Requerimientos Funcionales por Pantalla

4.1 GameListScreen (Tab: Catálogo)

Pantalla principal que muestra el catálogo de videojuegos disponibles en la tienda.

Conceptos:

- `useState`: Manejar el arreglo de videojuegos como estado local.
- `useEffect` con array vacío `[]`: Cargar videojuegos iniciales de ejemplo al montar el componente (simular datos obtenidos de una API de la tienda).
- `navigation.navigate` con parámetros: Al presionar una tarjeta de juego, navegar a `GameDetailScreen` enviando el objeto completo del videojuego.

Requisitos:

- Usar `FlatList` o `ScrollView` para renderizar el catálogo.
- Cada tarjeta debe mostrar al menos: título del juego, plataforma (PS5, Xbox, PC, Switch), género y un emoji representativo.
- Incluir un indicador visual de carga (`ActivityIndicator`) mientras el `useEffect` "obtiene" los datos.
- Estilos en archivo separado (`GameListStyles.js`) usando `StyleSheet.create()`.

4.2 GameDetailScreen (Stack desde Catálogo)

Pantalla de detalle que recibe los datos completos del videojuego seleccionado.

Conceptos:

- `route.params`: Recibir y desestructurar todos los datos del videojuego enviados desde `GameListScreen`.
- `useEffect` con dependencia: Ejecutar un efecto cada vez que cambie el parámetro recibido.
- `useState`: Manejar al menos un estado local adicional (por ejemplo, un toggle de "lista de deseos", una calificación con estrellas, o un contador de unidades a comprar).

Requisitos:

- Mostrar todos los campos del videojuego: título, plataforma, género, precio, clasificación de edad y descripción.
- Incluir un botón de "Volver"
- Diseño con tarjeta visual (sombras, bordes redondeados, colores por plataforma o género).
- Estilos en archivo separado (`GameDetailStyles.js`).

4.3 AddGameScreen (Tab: Agregar)

Formulario para agregar un nuevo videojuego al catálogo de la tienda.

Conceptos:

- `useState` (múltiples): Un estado independiente para cada campo del formulario (title, platform, genre, price, ageRating).
- Formulario controlado: Cada `TextInput` debe estar vinculado a su respectivo `useState` con `value` y `onChangeText`.
- `useEffect` con dependencias: Validar el formulario en tiempo real. Por ejemplo, habilitar/deshabilitar el botón de guardar según los campos estén completos y el precio sea un número válido.

Requisitos:

1. Mínimo 5 campos: título del juego, plataforma (PS5, Xbox, PC, Nintendo Switch), género (acción, aventura, RPG, deportes, estrategia), precio y clasificación de edad (E, T, M).
2. Usar `KeyboardAvoidingView` para evitar que el teclado tape los campos.
3. Botón de "Agregar Juego" que muestre un `Alert` con el resumen de los datos ingresados.
4. Botón de "Limpiar" que resetee todos los campos a su valor inicial.
5. Estilos en archivo separado (`AddGameStyles.js`).



Nota sobre persistencia

No se requiere persistencia real de datos. Es suficiente con que al presionar "Agregar Juego" se muestre el resumen en un `Alert`. Los videojuegos de ejemplo en `GameListScreen` pueden estar `hardcodeados` o simulados.

4.4 GamingNewsScreen (Tab: Noticias)

Pantalla que muestra noticias y novedades del mundo gaming con actualización automática.

Conceptos:

- `useEffect` con array vacío + `cleanup`: Crear un `setInterval` que rote automáticamente entre noticias gaming cada N segundos, y retornar la función de limpieza (`clearInterval`) para evitar `memory leaks`.
- `useState`: Manejar el índice de la noticia actual y el arreglo de noticias.
- `useEffect` con dependencia: Actualizar un mensaje informativo cada vez que cambie la noticia mostrada (por ejemplo, "Noticia 3 de 8").

Requisitos:

1. Mínimo 6 noticias gaming almacenadas en un archivo aparte (gamingNewsData.js). Ejemplos: lanzamientos próximos, actualizaciones de juegos, ofertas especiales, reseñas destacadas, eventos de esports, etc.
2. Rotación automática cada 5 segundos.
3. Botón manual para avanzar a la siguiente noticia.
4. Indicador visual de la noticia actual ("3 de 8").
5. Estilos en archivo separado (GamingNewsStyles.js).

5. Requisitos de Código y Buenas Prácticas

Criterio	Descripción
Idioma del código	Todo el código en inglés: variables, funciones, componentes, props, comentarios técnicos. La UI visible puede estar en español.
Indentación	2 espacios consistentes en todo el proyecto. Sin mezcla de tabs y espacios.
Nomenclatura	PascalCase para componentes (GameListScreen), camelCase para variables y funciones (gameTitle, handleSubmit), UPPER_SNAKE_CASE para constantes (MAX_NEWS_ITEMS).
Estilos separados	Cada pantalla debe tener su archivo de estilos en la carpeta styles/ usando StyleSheet.create().
Componentes funcionales	Solo componentes funcionales con hooks. No se permite el uso de componentes de clase.
Imports organizados	Primero dependencias externas (react, react-native), luego navegación, luego archivos locales (screens, styles, data).
Sin código muerto	No dejar imports sin usar, variables sin referenciar, ni bloques de código comentados.
SafeAreaView	Todas las pantallas deben estar envueltas en SafeAreaView como contenedor raíz.

6. Entrega

6.1 Repositorio Git

1. Crear un repositorio en GitHub (público o privado con acceso al docente).
2. Solo se revisará la rama main. Asegurarse de que el código final esté en esa rama.
3. El repositorio debe incluir un archivo README.md con: nombre del estudiante, descripción breve del proyecto, instrucciones para ejecutar la app

⚠ Atención

Commits con mensajes genéricos como "update", "fix", "changes" no serán considerados como buena práctica. Se espera que cada commit tenga un mensaje descriptivo en inglés. Ejemplo: "feat: add AddGameScreen with form validation".

6.2 Contenido del README.md

- Nombre completo del estudiante(s)
- Nombre de la aplicación (GameVault)
- Breve descripción del proyecto (2–3 líneas)
- Instrucciones de instalación y ejecución
- Lista de tecnologías utilizadas

8. Ejemplo de Referencia: useEffect con Cleanup

El siguiente fragmento ilustra el patrón esperado en GamingNewsScreen para la rotación automática de noticias con función de limpieza:

```
// GamingNewsScreen.js - Auto-rotating news with cleanup
const [currentIndex, setCurrentIndex] = useState(0);
const [news] = useState(GAMING_NEWS); // from gamingNewsData.js

// useEffect with empty array + cleanup function
useEffect(() => {
  const intervalId = setInterval(() => {
    setCurrentIndex((prev) => (prev + 1) % news.length);
  }, 5000);

  // Cleanup: clear interval when component unmounts
  return () => clearInterval(intervalId);
}, []);

// useEffect with dependency array
useEffect(() => {
  console.log(`Now showing news ${currentIndex + 1}`);
}, [currentIndex]);
```

¡Éxitos en el desarrollo del taller!

Recuerda: la clave es demostrar que entiendes cada concepto, no hacer uso de IAs.